

LAB VI: EVALUATE DIFFERENT ARITHMETIC OPERATIONS AND LOGICAL OPERATIONS ON TWO 32-BIT DATA USING ARM PROCESSOR

1. AIM OF THE EXPERIMENT

EVALUATE DIFFERENT ARITHMETIC OPERATIONS AND LOGICAL OPERATIONS ON TWO 32-BIT DATA USING ARM PROCESSOR

2. OBJECTIVES

1. Perform Addition and Subtraction of two 32-bit numbers using data processing addressing mode (with immediate data).
 2. Perform Addition, Subtraction, and Multiplication of two 32-bit numbers using load/store addressing mode.
 3. Perform the logical operations (AND, OR, XOR, and NOT) on two 32-bit numbers using load/store addressing mode.
-

3. THEORY

ARM Processor:

ARM (Advanced RISC Machines) is a 32-bit RISC processor with Load/Store architecture where instructions operate on registers only. Key features: 3-operand instructions, large register set, pipelined execution, and conditional execution capability.

Data Sizes: Byte (8 bits), Halfword (16 bits), Word (32 bits)

Registers:

- R0-R12: General purpose registers
- R13 (SP): Stack Pointer, R14 (LR): Link Register, R15 (PC): Program Counter
- CPSR: Current Program Status Register with flags N, Z, C, V
- SPSR: Saved Program Status Register

Condition Flags (CPSR):

N (Negative), Z (Zero), C (Carry), V (Overflow) - updated by arithmetic/logical operations

Addressing Modes:

1. **Data Processing:** Register (ADD R0, R1, R2), Immediate (ADD R3, R3, #1), Scaled Register (ADD R0, R0, R0, LSL #2)

2. **Load/Store:** Immediate offset (LDR R0, [R1, #4]), Register offset (LDR R0, [R1, R2]), Scaled register (LDR R0, [R1, R2, LSL #2])
3. **Branch:** 24-bit immediate value for target address
4. **Load/Store Multiple:** IA (Increment After), IB (Increment Before), DA (Decrement After), DB (Decrement Before)

Instruction Categories:

1. **Data Processing:** Arithmetic (ADD, SUB, MUL), Logical (AND, ORR, EOR, BIC), Move (MOV, MVN), Compare (CMP, CMN, TST, TEQ), Shift/Rotate (LSL, LSR, ASR, ROR)
2. **Data Transfer:** LDR/STR (word), LDRB/STRB (byte), LDRH/STRH (halfword), LDM/STM (multiple)
3. **Control Flow:** B (unconditional), BEQ/BNE (equal/not equal), BLT/BGT/BLE/BGE (comparisons), BL (branch with link)

4. OBSERVATION

Objective 1: Perform Addition and Subtraction of two 32-bit numbers using data processing addressing mode (with immediate data).

a) Pseudocode:

START

LOAD first 32-bit immediate value into register R0

LOAD second 32-bit immediate value into register R1

ADDITION:

ADD contents of R0 and R1

STORE the result in register R2

SUBTRACTION:

SUBTRACT contents of R1 from R0

STORE the result in register R3

DISPLAY results stored in registers R2 and R3

STOP

END

b) Code:

```
.global _start
_start:
    LDR R0, =0X230000AB
    LDR R1, =0X12000019
    ADD R2,R0,R1
    SUB R3,R0,R1
exit:b exit
```

c) Output:

The screenshot displays an ARMv7 assembly development environment. On the left, the 'Registers' window shows the state of various registers. On the right, the 'Editor (Ctrl-E)' window shows the assembly code being executed.

Register	Value
r0	230000ab
r1	12000019
r2	350000c4
r3	11000092
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000010
cpsr	000001d3 NZCVI SVC
spsr	00000000 NZCVI ?

```
1 .global _start
2 _start:
3     LDR R0, =0X230000AB
4     LDR R1, =0X12000019
5     ADD R2,R0,R1
6     SUB R3,R0,R1
7 exit:b exit
```

d) Analysis:

Input		Output	
Memory Location	Data	Memory Location	Data
-	0X230000AB	-	0X350000C4
-	0X12000019	-	0X11000092

Objective 2: Perform Addition, Subtraction, and Multiplication of two 32-bit numbers using load/store addressing mode.

a) Pseudocode:

START

INITIALIZE a memory pointer to the base address

LOAD first 32-bit number from memory into register R1

INCREMENT memory pointer to next memory location

LOAD second 32-bit number from memory into register R2

INCREMENT memory pointer to next memory location

ADDITION:

ADD contents of R1 and R2

STORE the result into memory

INCREMENT memory pointer

SUBTRACTION:

SUBTRACT contents of R2 from R1

STORE the result into memory

INCREMENT memory pointer

MULTIPLICATION:

MULTIPLY contents of R1 and R2

STORE the result into memory

STOP

END

b) Code:

```
.global _start
_start:
    LDR R0, =0X10100000
    LDR R1,[R0],#4
    LDR R2,[R0],#4
    ADD R3,R1,R2
    STR R3,[R0],#4
    SUB R4,R1,R2
    STR R4,[R0],#4
    MUL R5,R1,R2
    STR R5,[R0]
exit:b exit
```

c) Output:

Address	Memory contents and ASCII
100fff80	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100fff90	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100fffa0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100fffb0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100fffc0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100fffd0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100fffe0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100ffff0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100000	230000ab 12000019 350000c4 11000092
10100010	710010b3 0276000f aaaaaaaaaa aaaaaaaaaa
10100020	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100030	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100040	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100050	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100060	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100070	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100080	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100090	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
101000a0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
101000b0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
101000c0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
101000d0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa

d) Analysis:

Input		Output	
Memory Location	Data	Memory Location	Data
0X10100000	0X230000AB	0X10100008	0X350000C4
0X10100004	0X12000019	0X1010000C	0X11000092
-	-	0X10100010	0X710010B3
-	-	0X10100014	0X0276000F

Objective 3: Perform the logical operations (AND, OR, XOR, and NOT) on two 32-bit numbers using load/store addressing mode. a) Pseudocode:

START

INITIALIZE memory pointer to base address

LOAD first 32-bit number from memory into register R1

INCREMENT memory pointer to next memory location

LOAD second 32-bit number from memory into register R2

INCREMENT memory pointer to next memory location

AND OPERATION:

PERFORM bitwise AND on R2 and R1

STORE the result into memory

INCREMENT memory pointer

OR OPERATION:

PERFORM bitwise OR on R2 and R1

STORE the result into memory

INCREMENT memory pointer

XOR OPERATION:

PERFORM bitwise XOR on R2 and R1

STORE the result into memory

INCREMENT memory pointer

NOT OPERATION:

PERFORM bitwise NOT on R1

STORE the result into memory

STOP

END

b) Code:

```
.global _start
_start:
    LDR R0, =0x10100000
    LDR R1,[R0],#4
    LDR R2,[R0],#4
    AND R3,R2,R1
    STR R3,[R0],#4
    ORR R4,R2,R1
    STR R4,[R0],#4
    EOR R5,R2,R1
    STR R5,[R0],#4
    MVN R6, R1
    STR R6, [R0]
exit:b exit
```

c) Output:

Registers

Register	Value	Flags
r0	10100014	
r1	000000ab	
r2	000000ff	
r3	fffffff54	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	0000002c	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

Memory (Ctrl-M)

Go to address, label, or register: 10100000 Refresh »

Address	Memory contents and ASCII
100ffff80	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100ffff90	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100ffffa0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100ffffb0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100ffffc0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100ffffd0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100ffffe0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
100fffff0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100000	000000ab 000000ff 000000ab 000000ff
10100010	00000054 ffffffff54 aaaaaaaaaa aaaaaaaaaa
10100020	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100030	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100040	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100050	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100060	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100070	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100080	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
10100090	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
101000a0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
101000b0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
101000c0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa
101000d0	aaaaaaaa aaaaaaaaaa aaaaaaaaaa aaaaaaaaaa

Debugger Navigation: Registers, Call stack, Trace, Breakpoints, Watchpoints, Symbols, Counters, Settings

d) Analysis:

Input		Output	
Memory Location	Data	Memory Location	Data
0X10100000	0X000000AB	0X10100008	0X000000AB
0X10100004	0X000000FF	0X1010000C	0X000000FF
-	-	0X10100010	0X00000054
-	-	0X10100014	0XFFFFFFF54

5. CONCLUSION

Post Lab:

1. Give any examples of five arithmetic and logical instructions.
2. Differentiate between LDR and STR instruction.
3. Which of the following instructions is not valid a) MOV R7,R2 b) LDR R1, =LABEL
4. Explain briefly condition codes (flags) of ARM processor.

5. Which condition codes (flags) is considered for the following branch instructions?

B Label

BEQ label

BLT label

6. If the registers r1,r2,r3 contain the values 10,20,30 respectively, what will be the value in register r4 after the execution of the following code segment?

ADD r4, r1, r3

SUBS r4, r2, r4

RSB r4, r1, r4

